



**NayaHR**

AI-native HRIS · built for Indian businesses

# Technical Architecture & Data Flow

How the platform is built — the vendors, how they connect, and how data flows end to end.

---

*For founders & stakeholders · a walk-through for pitching and onboarding*



# What NayaHR is

An **AI-native HRIS** — a single platform that runs the entire people operation for **Indian small & mid-size businesses** (roughly 10–200 employees).



One clean platform, **multi-tenant by design** — every customer company is a fully isolated **tenant**, sharing infrastructure but never data.

7

core HR modules, one login

1 DB

many companies, isolated by row-level security

100%

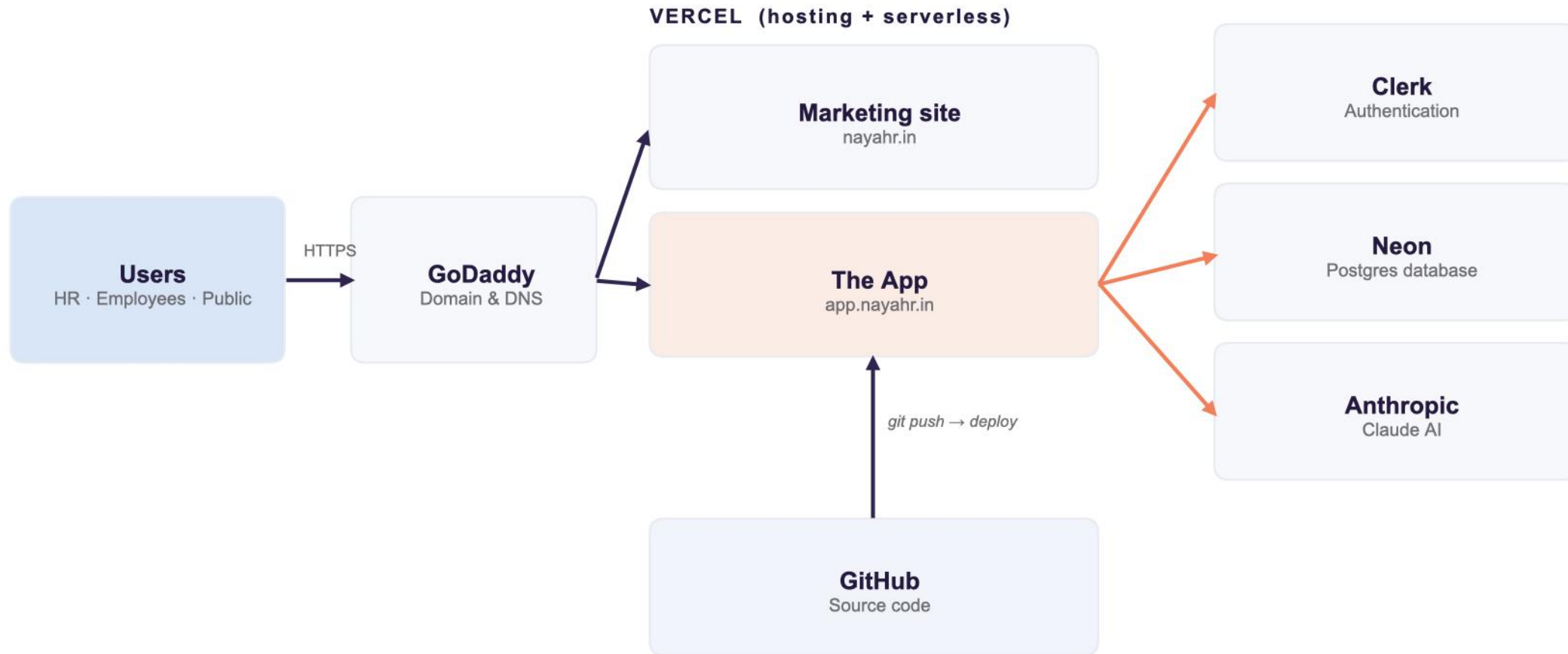
serverless — scales with usage

AI

assistant that gets work done



# The big picture



Every arrow is an HTTPS/API call. The App is the hub: it authenticates via Clerk, reads/writes tenant-isolated data in Neon, and calls Claude for AI actions. GoDaddy points the domains at Vercel; GitHub pushes deploy the code.



# The tech stack at a glance

| Vendor            | What it does                          | Why it's in the stack                                         |
|-------------------|---------------------------------------|---------------------------------------------------------------|
| GoDaddy           | Domain registrar & DNS                | Owns nayahr.in and points visitors to the right place         |
| Vercel            | Hosting, serverless compute & CI/CD   | Runs the Next.js app; auto-builds & deploys on every push     |
| Neon              | Serverless Postgres database          | Stores all data; enforces tenant isolation in the DB itself   |
| Clerk             | Authentication & user identity        | Sign-in, Google login, invitations — we never store passwords |
| Anthropic         | Claude AI models                      | Powers the assistant that reads and performs HR actions       |
| GitHub            | Source control & deploy trigger       | The code lives here; a push is what ships to production       |
| Next.js + Drizzle | App framework & typed database access | The application layer — UI, server logic, safe SQL            |



# Domains & DNS

One domain, three destinations. GoDaddy holds the DNS records that route each hostname to the right service:

**nayahr.in**

Marketing website

→ **Vercel (marketing project)**

**app.nayahr.in**

The HRIS application

→ **Vercel (platform project)**

**clerk.nayahr.in**

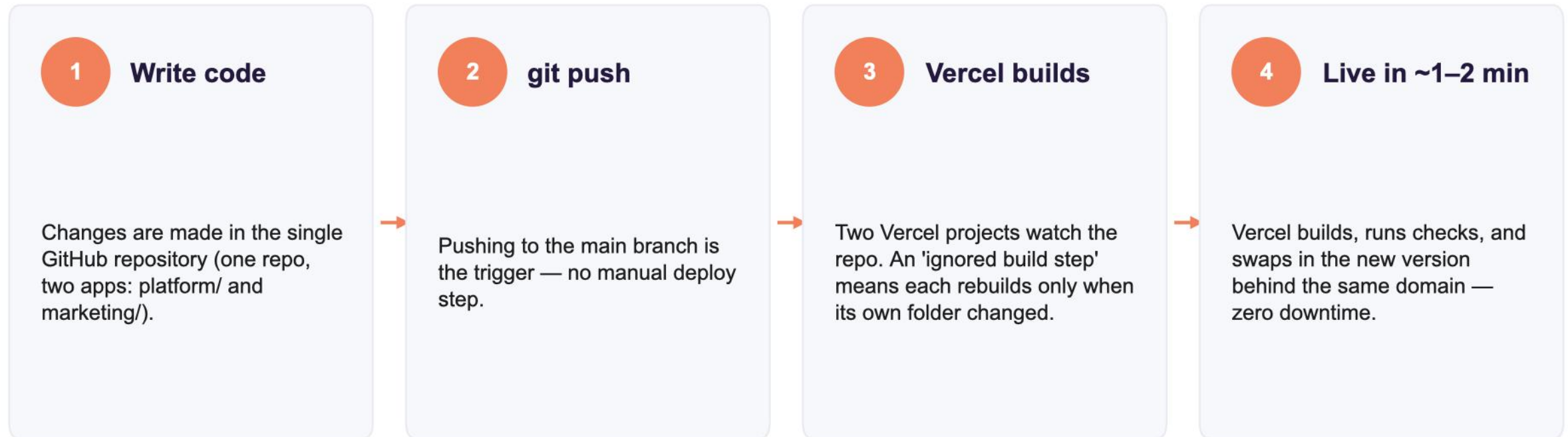
Authentication service

→ **Clerk (custom domain)**

*Records used: an A / CNAME record sends the apex + app subdomains to Vercel; a set of CNAMEs delegates clerk.nayahr.in (and email) to Clerk. SSL certificates are issued automatically.*



# The deployment pipeline



**Why two projects on one repo?** The marketing site and the app have different lifecycles, but sharing a repo keeps everything versioned together.





# Anatomy of a request

- 1 User opens app.nayahr.in** GoDaddy DNS resolves the hostname to Vercel, which serves the Next.js app.
- 2 Clerk checks identity** Middleware asks Clerk 'is this person signed in?' If not, they're sent to sign-in (Google or email).
- 3 Invite-only gate** A signed-in but un-invited account is stopped — no access without an invitation.
- 4 Session is resolved** getSession() maps the Clerk user → their app\_user record → their company (tenant) and role.
- 5 Query runs under RLS** Every DB call sets an 'app.tenant' marker; Postgres row-level security auto-filters to that company's rows only.
- 6 Page renders on the server** People, payroll, etc. are fetched and rendered server-side, then sent to the browser.
- 7 AI actions, same rules** The assistant calls Claude with role-scoped tools that go through the very same RLS-guarded data layer.



# Authentication & identity — Clerk

## Sign-in methods

Email + password and one-click Google sign-in, handled entirely by Clerk.

## No passwords, ever

We never see or store credentials — Clerk holds them, reducing our security surface dramatically.

## Invitations

When HR invites an employee, the app calls Clerk to send a real sign-up invitation email.

## Restricted sign-up

Clerk is set to invite-only mode: nobody can create a login unless they were invited.

## Identity mapping

Each Clerk user is linked to an 'app\_user' row, which ties them to their company and role.

## Own domain

Clerk runs on clerk.nayahr.in, so the whole experience stays on the NayaHR brand.





# Multi-tenancy & security: Row-Level Security

1

## One database, many companies

All customers share one Postgres database. Every row is stamped with a `tenant_id`.

2

## The database enforces isolation

Postgres Row-Level Security policies auto-filter every query to the current company — set once per request via a secure session marker.

3

## It can't be bypassed

RLS is FORCED, and the app connects with a dedicated role that is NOT allowed to skip it (we specifically fixed a default that could).

***“A bug in our app code can't leak another company's data — the database itself refuses to return it.”***

Security enforced at the lowest layer — not just in app code.

*Defense in depth: identity (Clerk) → tenant scoping (session) → row-level security (Postgres) → full audit log of every change.*



# The data model — effective-dated

Records aren't overwritten — they're **append-only and effective-dated**. Every job or pay change is a new dated event, so we can reconstruct the org **as of any date** and keep a complete history.

|                                |                      |                              |                                                  |
|--------------------------------|----------------------|------------------------------|--------------------------------------------------|
| <b>tenant</b>                  | A customer company   | <b>app_user</b>              | A login + role (owner / HR / manager / employee) |
| <b>worker</b>                  | An employee record   | <b>job_event</b>             | Dated job history (title, dept, manager, status) |
| <b>compensation_event</b>      | Dated salary history | <b>payroll_run · payslip</b> | Monthly runs + immutable payslips                |
| <b>requisition · candidate</b> | Recruitment pipeline | <b>review · goal</b>         | Performance cycle                                |
| <b>leave_request</b>           | Time off             | <b>audit_log</b>             | Every change, who and when                       |



# The modules



## Core HR

Employee records, org, effective-dated changes with approvals



## Recruitment

Requisitions → candidate pipeline → offer → hire into Core HR



## Performance

Goals & review cycle: self → manager → HR



## Compensation

Salary structure, dated change history & approvals



## Payroll

India statutory payroll: PF, ESI, PT, TDS → payslips



## Time off

Indian leave types, balances, approvals, loss-of-pay



## Reporting

Headcount, cost and people analytics



## AI assistant

Natural-language actions across every module



# Inside the payroll engine



## Statutory calculations (India, built in):

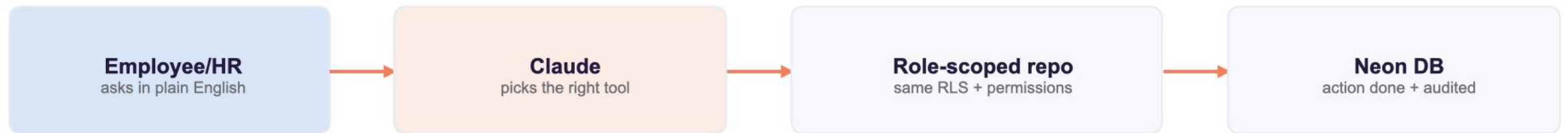
|                                            |                                                  |                                                 |
|--------------------------------------------|--------------------------------------------------|-------------------------------------------------|
| <b>Provident Fund (PF)</b><br>12% of basic | <b>ESI</b><br>0.75% / 3.25% up to ₹21,000 gross  | <b>Professional Tax</b><br>state slab (₹200/mo) |
| <b>TDS</b><br>estimated, new tax regime    | <b>Loss of Pay</b><br>prorated from unpaid leave | <b>Employer cost</b><br>PF + ESI added on top   |

*Finalized payslips are an immutable snapshot — they never change even if salary later moves. Pulls live from Compensation and Time off.*



# The AI assistant — Anthropic Claude

The assistant uses **Claude** with a set of **tools** — it doesn't just chat, it performs real actions by calling the same secure data layer the UI uses.



## Examples

- *“Create a requisition for a Senior Engineer in Bengaluru.”*
- *“Run July payroll.”*
- *“Raise a compensation change for Priya, effective next month.”*
- *“How many people are on leave this week?”*

### Safe by construction

The AI can only do what the signed-in user's role allows — it goes through the same permission checks and tenant isolation as every click in the UI.





# Access & the invite-only model

## Owner

The account owner. Everything, plus managing admins & billing. One per company.

## HR Admin

Runs day-to-day HR across the whole company — people, hiring, pay, approvals.

## Manager

Sees and acts on their own team; raises changes that HR approves.

## Employee

Self-service: their own profile, payslips, leave and goals.

## Invite-only, two layers deep

**1) App gate:** a signed-in account with no invitation gets a 'you need an invite' wall — no company is ever auto-created. **2) Clerk restricted mode:** non-invited people can't even create a login.



# Security, audit & compliance

## Tenant isolation in the DB



Row-level security means one company can never query another's data — enforced by Postgres, not just app code.

## Managed authentication



Clerk handles logins & sessions; we never store passwords. Google OAuth + invite-only sign-up.

## Full audit trail



Every create, change and approval is written to an append-only audit log — who, what, when.

## Secrets stay out of code



Database URLs and API keys live only in the Vercel / Neon / Clerk dashboards, never in the repo.

## Least-privilege DB role



The app connects with a role that cannot bypass security — a deliberate hardening step.

## Built for India (DPDP-aware)



Architecture supports data-residency and privacy needs of the Indian DPDP Act as we formalize policy.



# Scaling & cost model

Everything is **serverless and usage-based** — near-zero fixed cost, and it scales automatically as companies and employees are added. No servers to run.

**Vercel****Per usage / seat**

App hosting &amp; serverless functions — scales on traffic

**Neon****Per compute + storage**

Database; connection pooling keeps it efficient at scale

**Clerk****Per monthly active user**

Auth cost tracks the number of people actually logging in

**Anthropic****Per token (AI usage)**

Only pay when the AI assistant is actually used

*Net effect: cost rises roughly in step with paying customers — healthy unit economics, and a tiny team can operate the whole platform.*



# Why this architecture wins

## Secure by design

Tenant isolation enforced in the database — trust you can defend to any buyer.

## AI-native

An assistant that performs real work, safely, within each user's permissions.

## Serverless & lean

Best-in-class managed vendors; a small team ships fast with near-zero ops.

## Built for India

Statutory payroll, Indian leave, and DPDP-aware from the ground up.